Backend technical reference

Updated November 2, 2026 for schema `0005_protected_workflows`. This guide explains the implemented behavior. The algorithm reference provides pseudocode and the formula reference defines scores.

1. What the backend does

The backend turns a reviewed engagement into individually approved scanner steps, bounded evidence, analyzed findings and versioned reports. A configured provider can assist planning and analysis; deterministic fallbacks remain available. Neither provider output nor autopilot grants authorization.

The normal phase sequence is scoping, recon, vulnerability analysis, exploitation, post-exploitation and reporting. Aggressive lab mode extends the plan through credential validation, lateral movement and impact. Some phases are bookkeeping states rather than separate scanner plans.

2. Components

Module	Responsibility
`main.py`, `workspace_api.py`	Authenticated API, setup, workflow, paginated reads and reviews
`database.py`, `migrations/`	Models, SQLAlchemy sessions and Alembic upgrades
`models.py`	Request validation and size bounds
`api_auth.py`	Operator-key validation and generated-key digests
`planner.py`, `exploit_planner.py`	Recon and evidence-based verification plans
`planners.py`, `footholds.py`	Campaign plans, references and evidence ingestion
`policy_engine.py`, `scope_rules.py`, `command_values.py`	Tool, argument, destination and engagement checks
`jobs.py`, `worker.py`	Durable jobs, claims, heartbeats and recovery
`executor.py`, `ssh_executor.py`	Local process or trusted SSH/tmux execution
`evidence.py`, `secret_store.py`	Encrypted originals, public redaction and legacy adoption
`workflow.py`	Revision claims and post-commit report-file retirement
`analyzer.py`, `verification.py`	Findings/scoring and independent confirmation
`next_steps.py`, `auto_recommend.py`	Bounded proposals and opt-in autopilot
`attack_paths.py`, `provider.py`	Finding correlations and provider transport
`reporter.py`, `report_versions.py`	HTML rendering, immutable files and provenance
`observability.py`, `logging_config.py`	Request timing and structured logging

See the API reference for all application routes. Health and readiness probes are unauthenticated; capabilities, data and mutations require the operator key. FastAPI schema/documentation endpoints are also public.

3. Persistent data

SQLite defaults to `data/redteam.db`, relative to the backend working directory. Production connections enable foreign keys, a busy timeout and WAL. Production sessions use `autoflush=False`; services explicitly flush when later steps in a transaction need generated IDs or new evidence.

Table	Stored purpose
`targets`	Scope, restrictions, criticality, policy and exploitation/lab flags
`app_settings`	One configuration row; secret fields are encrypted
`assessments`	Plan, phase, analyzed phases, metadata, parent and autopilot state
`tool_executions`	Current step attempt, job lease, redacted output and encrypted original
`execution_attempts`	Completed attempt snapshots with separate encrypted originals
`findings`	Scores, location, evidence, originating phase and verification
`finding_reviews`	Owner, due date, status, justification, comments and retest evidence
`recommendations`	Proposal batches and durable original-access audit records
`recommendation_tasks`	Per-execution/attempt recommendation processing state
`report_versions`	Immutable report path, digest, provenance and restricted originals
`footholds`	Evidence-derived sessions/logins and their state
`loot`	Encrypted credentials, fingerprints and account/service metadata
`engagement_hosts`	Discovered/owned hosts and child-assessment links

An assessment has both `plan_version` and `workflow_revision`. The former coordinates operator plan review; the latter protects state publication across API and worker operations. `current_report_version_id` identifies the current public report. `evidence_protected` records completion of legacy backfill.

4. Startup and upgrades

FastAPI lifespan runs database initialization before serving requests. Alembic adopts older schemas and upgrades through `0005_protected_workflows`; settings secret migration and evidence backfill follow. A standalone external worker waits for the expected schema and for all legacy assessment evidence to be adopted before taking jobs.

Existing execution and attempt originals become encrypted, while derived public text is redacted. Legacy report bytes and digests are retained behind audited access; their public files are retired after commit. Regeneration produces a new redacted report. Keep the original master key and upgrade API and worker together. See upgrade details.

5. Plan editing and scope

A supplied or generated plan is bounded to 50 steps. The default host plan has eight steps: Nmap, traceroute, DNS lookup, curl headers, WhatWeb, SSLScan, passive ZAP baseline and Nuclei. Restricted tools or policy filtering may reduce it. CIDR targets use two Nmap sweeps; primary networks default to at most 256 addresses.

Executed steps retain their index, tool, command, phase and enabled state. Unexecuted work remains editable while idle. Execution-bearing changes invalidate the earliest changed phase and its dependents, preserving earlier analysis. An unchanged save keeps analysis/report; rationale edits invalidate only the current report. Recon is drafted at creation, so recon-redraft requests return a controlled conflict.

At approval and again when a worker starts, the backend checks phase prerequisites, letter restrictions, engagement window, capabilities, command arguments, all recognized destinations and exclusions. Rate limits apply to supported scanner options; the framework is not a universal traffic shaper.

6. Execution and recovery

An explicit approval claims the assessment and creates or reuses its unique step-execution row. The worker conditionally claims queued work, resolves loot and foothold references, revalidates policy and starts the selected executor.

The local executor uses parsed arguments with `create_subprocess_exec`, without a general shell. The VM executor uses SSH/tmux with literal argument quoting and requires a trusted host-key fingerprint. Both have output and time bounds; a native Python install does not provide scanner binaries.

Browser disconnects do not cancel durable jobs. Cancellation is explicit. Workers heartbeat while running; stale leases become interrupted and require re-approval. A late worker cannot overwrite an attempt it no longer owns. Each completed attempt is archived before a retry replaces the current projection.

A zero exit is generally completed. ZAP baseline exits 1 and 2 mean completed-with-findings; other nonzero results are failures unless a cancellation, interruption or timeout state applies. A failed scan is not a clean target.

7. Protected evidence

The approved command and captured stdout/stderr have an encrypted JSON original. Public execution fields, attempt snapshots, findings and report evidence use redacted projections. Original stdout/stderr hashes remain available for provenance. Internal campaign parsing and independent verification can read originals; provider analysis receives redacted input.

Redaction recognizes supported credential tables/lines, hashes, labeled credentials/tokens, private keys, URL credentials and known recovered values. It is not a complete detector for arbitrary unlabelled secrets. Structured workflow/credential identity fields are preserved rather than rewritten as text.

The live registry retains a bounded beginning of the raw stream privately so table headers remain available. Only complete redacted lines are public; unfinished/overlong lines are withheld. Completed execution storage follows the executor's output cap.

Known recovered plan credentials use `$$loot:<id>$$` references. Executable operator-authored plan source is not a secret vault; use references rather than typing new credentials. Public execution redaction does not alter the actual approved command.

Original and loot reveals require operator authentication, commit an `evidence_access` audit record, and return `Cache-Control: no-store`. Wrong/missing encryption keys produce a conflict, not an empty replacement original. Settings credentials retain their separate legacy unreadable-as-unset behavior.

8. Analysis, verification and publication

Analysis requires terminal results for every enabled step of the current phase. It captures the starting revision and attempts, computes results, then claims the matching revision before writing. Findings, independent verification, campaign ingestion, phase bookkeeping and eligible automatic drafting commit together. An exception rolls back that operation.

Deterministic parsers cover Nmap, Nuclei, curl, WhatWeb, SSLScan, sqlmap, Searchsploit, Metasploit and passive ZAP output. Driver bounds, deduplication and rescoring also apply to provider findings. Failed tools are disclosed separately.

The confirmation adapter recognizes explicit successful sqlmap injection proof. A banner, HTTP 200, exploit lookup or model-authored statement does not establish verification. Links must match vulnerability category, endpoint and parameter; the endpoint check preserves path/query distinctions.

Finding risk is exploitability times impact times exposure (1-125). Priority is a weighted 0-100 score using severity, exploitability, target criticality and confidence. These are framework scores, not CVSS or probabilities.

9. Campaign and autopilot

Parsers record observed hosts, sessions/logins and recovered credentials; they do not infer compromise from mere discovery. Loot is encrypted and deduplicated by credential identity/fingerprint. Foothold and credential references are resolved just before execution.

Aggressive-lab operations require both exploitation permission and lab mode. Promotion additionally requires owned state, host-wide permission in the parent's scope and no exclusion overlap. URL/path/port-only scopes cannot authorize a whole-host child plan. A revision claim prevents duplicate concurrent promotion; parent restrictions are copied to the child.

Autopilot is opt-in and bounded to 25 iterations. Its local ingestion/draft/ bookkeeping transition is atomic. Advisory provider work runs after that commit, outside the write lock, and is stored only if its revision is still current. It never approves commands or runs analysis/execution on the operator's behalf.

10. Reports and deletion

Rendering uses a unique staged file. Publication verifies the assessment revision, archives an immutable digest-named report and commits the current report pointer. Stale rendering returns 409 and cleans staging. Existing public versions remain available through versioned routes.

The mutable `report_{id}.html` path is no longer the current-report contract. Downloads resolve the committed database pointer. Invalidating current output does not rewrite historical digests. Restricted legacy originals are served only as audited attachments.

Deletion respects campaign foreign-key order. A parent with child assessments cannot be deleted until its children are removed. Reset clears target, assessment and evidence data while preserving settings; active work blocks destructive cleanup.

11. Bounds and validation

Bound	Default
Uploaded document	5 MiB
Command timeout	360 seconds
Captured stdout/stderr	200,000 characters each after bounded byte retention
Live raw buffer	200,000 characters
Plan length	50 steps
Primary CIDR target	256 addresses
Analyzer findings	200
Per-finding text	20,000 characters
Analysis stream / aggregate budget	120,000 / 400,000 characters
Default target criticality	70 / 100
Footholds / loot / hosts	50 / 200 / 512 per assessment
Autopilot iterations	25
Default execution concurrency	2, configurable within 1-8

The testing guide records the 412-test full backend run, subsequent focused checks and 24 frontend tests. Those checks used isolated storage and mocked executors. Live tool compatibility and Docker deployment are separate validation tasks. The shared SSH pane also needs cross-process ownership before multiple workers can safely use it.